



Práctica 1C

Práctica clusters

Aprendizaje No Supervisado y Visualización
Unidad 1 · Maestría en Ciencia de Datos e Información

Bryan Rodríguez Abarca

Matrícula 26DF1185A9

3 de septiembre de 2026

X.npy trae 1797 datos y los cuatro algoritmos corren sobre ese mismo arreglo. Con los valores por omisión DBSCAN marca los 1797 como ruido, así que el trabajo estuvo en mover parámetros: k-means, DBSCAN, HDBSCAN y AgglomerativeClustering, cada uno calificado con la métrica intra/inter de 1B y con la silueta. Al final comparo si las dos coinciden en cuál algoritmo agrupa mejor.

```
import numpy as np
import sklearn, sys

print("python      ", sys.version.split()[0])
print("numpy       ", np.__version__)
print("scikit-learn", sklearn.__version__)
```

```
python      3.11.15
numpy       2.4.6
scikit-learn 1.9.0
```

1. Los datos

X es el mismo arreglo de la práctica 1B: 1797 datos de 64 columnas con valores entre 0 y 1, que en 1B identifiqué como `load_digits` con todo dividido entre 16. Eso me da una expectativa contra la cual leer lo que devuelva cada algoritmo: si los datos son diez dígitos, un método que parta el conjunto en dos o en cuarenta grupos está diciendo algo raro.

```
from sklearn.cluster import KMeans
from sklearn.cluster import DBSCAN
from sklearn.cluster import HDBSCAN
from sklearn.cluster import AgglomerativeClustering
import numpy as np
from sklearn.metrics import silhouette_score

X = np.load("X.npy")
X.shape
```

```
(1797, 64)
```

2. La métrica y las etiquetas -1

Uso la función que implementé en la práctica 1B: muestrea 500 pares de índices con `np.random.seed(0)`, separa los pares que comparten etiqueta de los que no, y devuelve el cociente entre la distancia media de unos y la de otros. Valores chicos son mejores. La silueta va al revés: mejor cuanto más alta.

Sobre las etiquetas -1 que devuelven DBSCAN y HDBSCAN mantengo la decisión que tomé en 1B: la etiqueta se compara tal cual, de modo que dos datos marcados con -1 cuentan como pareja del mismo grupo. La silueta también las recibe tal como salen del algoritmo.

Como referencia de qué significa “malo”, reparto los 1797 datos en nueve grupos al azar y califico ese reparto.

```
def intra_inter(X, y, num_pairs):
    np.random.seed(0)
    pairs = np.random.randint(len(X), size = (num_pairs, 2))
    a = X[pairs[:, 0]]
    b = X[pairs[:, 1]]
    d = np.sqrt(((a - b) ** 2).sum(axis=1))
    mismo = y[pairs[:, 0]] == y[pairs[:, 1]]
    return d[mismo].mean() / d[~mismo].mean()

def calidad(y):
    return intra_inter(X, y, 500), silhouette_score(X, y)

def n_grupos(y):
    # el ruido va marcado con -1 y los grupos de 0 en adelante
    return len(np.unique(y[y >= 0]))

np.random.seed(1)
azar = np.random.randint(9, size=len(X))
print("reparto al azar en 9 grupos:  intra/inter %.4f  silueta %.4f"
      % calidad(azar))
```

reparto al azar en 9 grupos: intra/inter 0.9822 silueta -0.0186

3. k-means

Barro `n_clusters` de 2 a 50 y califico cada partición con las dos métricas. La inercia va en la tabla porque es lo que k-means minimiza y me deja ver que las corridas no se quedaron en soluciones distintas entre sí.

```
ks = [2, 4, 6, 8, 9, 10, 11, 12, 14, 16, 20, 30, 50]

print(" k  inercia  intra/inter  silueta")
for k in ks:
    km = KMeans(n_clusters=k, n_init=10, random_state=0).fit(X)
    a, s = calidad(km.labels_)
    print(f"{k:3d}  {km.inertia_:7.1f}  {a:11.4f}  {s:7.4f}")

sil = {}
print()
print("silueta en 20 semillas de inicialización")
print(" k      mínima      media      máxima")
```

```

for k in [9, 14]:
    sil[k] = [silhouette_score(X, KMeans(n_clusters=k, n_init=10,
        random_state=r).fit_predict(X)) for r in range(20)]
    print(f"{k:3d}  {min(sil[k]):8.4f}  {np.mean(sil[k]):8.4f}"
          f"  {max(sil[k]):8.4f}")
print("  semillas en que k = 14 supera a la peor de k = 9:",
      sum(1 for v in sil[14] if v > min(sil[9])))

```

k	inercia	intra/inter	silueta
2	7479.0	0.9104	0.1183
4	6297.9	0.8292	0.1229
6	5488.2	0.7823	0.1515
8	4941.6	0.7402	0.1784
9	4696.5	0.7189	0.1893
10	4551.5	0.7108	0.1825
11	4421.1	0.6787	0.1830
12	4345.2	0.6927	0.1848
14	4076.3	0.6788	0.1876
16	3933.2	0.6529	0.1716
20	3663.8	0.6409	0.1734
30	3280.3	0.6125	0.1467
50	2807.1	0.5736	0.1339

silueta en 20 semillas de inicialización

k	mínima	media	máxima
9	0.1890	0.1892	0.1894
14	0.1715	0.1848	0.1890

semillas en que k = 14 supera a la peor de k = 9: 0

Mi primer intento fue ordenar los barridos por la propia intra/inter, y me pedía $k = 50$: la columna baja en todos los pasos menos el de $k = 12$, y en $k = 50$ da el valor más bajo de la tabla con la silueta ya caída a 0.1339. Cincuenta grupos para un conjunto de diez dígitos no me sirven, así que ordeno por silueta y reporto las dos métricas sobre la partición que esa elección deje. La silueta elige y la intra/inter solo mide, y lo tengo en cuenta al responder la pregunta del final.

La silueta más alta del barrido está en $k = 9$ con 0.1893, y $k = 14$ queda a 0.0017 de ahí. Como los dos salen de una inicialización aleatoria, repetí ambos con 20 semillas: $k = 9$ se mantiene entre 0.1890 y 0.1894, mientras que $k = 14$ baja hasta 0.1715 y promedia 0.1848, sin superar en ninguna de las 20 a la peor corrida de $k = 9$. Me quedo con $k = 9$.

4. AgglomerativeClustering

El mismo barrido de `n_clusters`, y además los cuatro tipos de enlace que acepta la clase, evaluados en el k que resultó mejor.

```

print(" k intra/inter silueta")
for k in [2, 4, 6, 8, 9, 10, 11, 12, 14, 16, 20]:
    y = AgglomerativeClustering(n_clusters=k).fit_predict(X)
    a, s = calidad(y)
    print(f"{k:3d} {a:11.4f} {s:7.4f}")

print()
print("tipos de enlace con k = 9")
for enlace in ["ward", "complete", "average", "single"]:
    y = AgglomerativeClustering(n_clusters=9,
                                linkage=enlace).fit_predict(X)
    a, s = calidad(y)
    print(f" {enlace:9s} intra/inter {a:.4f} silueta {s:7.4f}"
          f" grupo mayor {np.bincount(y).max():4d}")

```

k	intra/inter	silueta
2	0.9096	0.1167
4	0.8726	0.1090
6	0.8241	0.1370
8	0.7649	0.1706
9	0.7327	0.1806
10	0.7220	0.1785
11	0.7220	0.1764
12	0.7162	0.1736
14	0.6855	0.1767
16	0.6653	0.1770
20	0.6534	0.1686

tipos de enlace con k = 9

enlace	intra/inter	silueta	grupo mayor
ward	0.7327	0.1806	317
complete	0.7888	0.1138	398
average	0.7826	0.1301	480
single	0.9913	-0.1324	1789

La silueta sube hasta $k = 9$, con 0.1806, salvo un bache en $k = 4$ que la baja de 0.1167 a 0.1090 antes de volver a subir. Los dos métodos que reciben el número de grupos como parámetro terminan pidiendo el mismo.

ward, que es el enlace por omisión, gana en las dos métricas. **single** sirve para ver qué tan mal puede salir esto: amontona 1789 de los 1797 en un grupo único y reparte los ocho restantes, con lo que la silueta se va a -0.1324, por debajo del reparto al azar.

5. DBSCAN

eps es el radio con el que DBSCAN decide si dos datos son vecinos, y no tenía idea de en qué escala pedirlo. La forma usual de elegirlo es mirar las distancias de cada dato a su k -ésimo vecino más cercano y buscar el codo; la probé y me sirvió para acotar el barrido, aunque aquí el codo no

se ve tan claro como para leer un valor de él. `kneighbors` cuenta el propio punto en la columna 0, así que la columna del quinto vecino son cuatro puntos distintos, que es también como cuenta `min_samples`.

```
from sklearn.neighbors import NearestNeighbors

y = DBSCAN().fit_predict(X)
print("valores por omisión (eps = 0.5, min_samples = 5):")
print("    etiquetas distintas", np.unique(y),
      "    datos sin asignar", (y == -1).sum())

print()
print("distancia al k-ésimo vecino, contando el propio punto")
print("  k      p25      p50      p75      p90")
for k in [5, 10, 20, 30]:
    vec = NearestNeighbors(n_neighbors=k).fit(X)
    dk = vec.kneighbors(X)[0][:, -1]
    p25, p50, p75, p90 = np.percentile(dk, [25, 50, 75, 90])
    print(f"{k:3d}  {p25:6.2f}  {p50:6.2f}  {p75:6.2f}  {p90:6.2f}")

print()
print("eps  min_samples  grupos  ruido  intra/inter  silueta")
for ms in [5, 10, 20, 30]:
    for eps in [1.3, 1.4, 1.5, 1.6, 1.7]:
        y = DBSCAN(eps=eps, min_samples=ms).fit_predict(X)
        a, s = calidad(y)
        print(f"{eps:.1f}  {ms:11d}  {n_grupos(y):6d}  "
              f"{(y == -1).sum():5d}  {a:11.4f}  {s:7.4f}")

print()
print("arriba de 1.7, con min_samples = 20:")
for eps in [1.8, 2.0, 2.2, 2.5]:
    y = DBSCAN(eps=eps, min_samples=20).fit_predict(X)
    print(f"  eps = {eps}    grupos {n_grupos(y)}"
          f"    ruido {(y == -1).sum()}")
```

```
valores por omisión (eps = 0.5, min_samples = 5):
    etiquetas distintas [-1]    datos sin asignar 1797
```

```
distancia al k-ésimo vecino, contando el propio punto
  k      p25      p50      p75      p90
  5      1.10      1.24      1.39      1.57
 10      1.25      1.41      1.58      1.77
 20      1.42      1.60      1.77      1.95
 30      1.54      1.71      1.89      2.08
```

```
eps  min_samples  grupos  ruido  intra/inter  silueta
```

1.3	5	23	346	0.7933	0.1000
1.4	5	11	201	0.8447	0.0699
1.5	5	8	106	0.9087	0.0168
1.6	5	2	35	0.9877	0.0037
1.7	5	1	15	0.9758	0.0583
1.3	10	13	661	0.9090	0.0804
1.4	10	13	385	0.7863	0.1041
1.5	10	10	201	0.8315	0.0863
1.6	10	2	90	0.9747	0.0215
1.7	10	1	39	0.9935	0.0467
1.3	20	8	1210	1.0011	-0.0440
1.4	20	10	792	0.9313	0.0694
1.5	20	9	467	0.8421	0.1440
1.6	20	5	231	0.9137	0.0591
1.7	20	4	100	0.9356	0.0133
1.3	30	3	1454	0.9985	0.0032
1.4	30	6	1228	1.0170	-0.0126
1.5	30	10	710	0.9028	0.0866
1.6	30	8	460	0.8491	0.1427
1.7	30	3	216	0.9154	0.0625

arriba de 1.7, con `min_samples = 20`:

```
eps = 1.8  grupos 2  ruido 45
eps = 2.0  grupos 1  ruido 3
eps = 2.2  grupos 1  ruido 0
eps = 2.5  grupos 1  ruido 0
```

Con los valores por omisión DBSCAN no encuentra nada: los 1797 datos salen marcados como ruido y la única etiqueta es -1, así que ni siquiera se le puede pedir la silueta. La tabla de vecinos explica por qué. La mitad de los datos necesita recorrer más de 1.24 para juntar cinco vecinos contándose a sí mismo, y `eps` viene en 0.5.

Esa misma tabla me dio la escala del barrido: la mediana de la distancia al vecino 20 es 1.60, así que probé `eps` entre 1.3 y 1.7 contra cuatro valores de `min_samples`. La ventana útil es angosta. Por arriba, en 1.8 ya solo quedan dos grupos y 45 datos fuera, y en 2.0 un grupo único con todo adentro; por abajo, en 1.3 con `min_samples=30` se van 1454 de los 1797 datos al ruido y el cociente se pega a 1.

Dentro de la ventana las dos métricas ya no señalan lo mismo. La silueta más alta, 0.1440, sale con `eps=1.5` y `min_samples=20`; la intra/inter más baja, 0.7863, sale con `eps=1.4` y `min_samples=10`, donde la silueta cae a 0.1041. Por el criterio que fijé arriba corro DBSCAN con `eps=1.5` y `min_samples=20`, y con eso 467 de los 1797 se quedan fuera de todo grupo, arriba del 25 %. La única combinación del barrido que baja ese número sin perder silueta es `eps=1.6` con `min_samples=30`: siete datos menos y 0.0013 de silueta menos, que no cambia nada.

6. HDBSCAN

Aquí el parámetro que manda es `min_cluster_size`, el tamaño mínimo que debe tener algo para llamarse grupo. Le agrego a la tabla el tamaño mediano y el del grupo mayor, porque con solo el

número de grupos y el ruido no alcanzaba para entender qué estaba pasando al subirlo.

```
print("min_cluster_size grupos ruido mediana mayor "
      "intra/inter silueta")
for mcs in [3, 5, 8, 10, 15, 20, 30, 50]:
    # copy=True: sklearn avisa que ese valor por omisión cambia
    y = HDBSCAN(min_cluster_size=mcs, copy=True).fit_predict(X)
    a, s = calidad(y)
    tam = np.bincount(y[y >= 0])
    print(f"{mcs:16d} {n_grupos(y):6d} {(y == -1).sum():5d} "
          f"{np.median(tam):7.0f} {tam.max():5d} {a:11.4f} {s:7.4f}")

print()
print("mcs min_samples grupos ruido intra/inter silueta")
for mcs in [5, 8, 15]:
    for ms in [3, 5, 10]:
        y = HDBSCAN(min_cluster_size=mcs, min_samples=ms,
                     copy=True).fit_predict(X)
        a, s = calidad(y)
        print(f"{mcs:3d} {ms:11d} {n_grupos(y):6d} "
              f"{(y == -1).sum():5d} {a:11.4f} {s:7.4f}")
```

min_cluster_size	grupos	ruido	mediana	mayor	intra/inter	silueta
3	42	418	6	177	0.8036	0.0423
5	21	497	35	176	0.8202	0.0719
8	15	723	42	174	0.9110	0.0470
10	12	816	73	173	0.9162	0.0392
15	10	894	91	171	0.9310	0.0294
20	8	914	124	168	0.9589	0.0472
30	7	1021	114	161	0.9768	0.0297
50	2	984	406	745	0.9757	0.0005

mcs	min_samples	grupos	ruido	intra/inter	silueta
5	3	27	404	0.8046	0.0747
5	5	21	497	0.8202	0.0719
5	10	15	802	0.9200	0.0213
8	3	19	397	0.7851	0.0924
8	5	16	490	0.8319	0.0970
8	10	13	807	0.9159	0.0375
15	3	14	424	0.8187	0.1160
15	5	13	523	0.8423	0.1034
15	10	11	800	0.9177	0.0538

Moviendo solo `min_cluster_size` lo mejor que consigo es 0.0719 de silueta con 5, y de ahí en adelante empeora. Lo que pasa al pedir grupos más grandes no es que los grupos crezcan de un lado: entre 5 y 30 el grupo mediano pasa de 35 a 114 datos porque los chicos desaparecen, mientras que el mayor baja de 176 a 161, y el ruido sube de 497 a 1021. Buscando por qué, el detalle que lo explica

es que `min_samples` toma por omisión el mismo valor que `min_cluster_size`, y es `min_samples` quien decide qué tan lejos considera “denso” el algoritmo. Las dos tablas lo enseñan: el renglón de `min_cluster_size=5` con `min_samples=5` da 0.8202 y 0.0719, los mismos números del renglón de arriba donde no fijé `min_samples`.

Al separarlos el resultado cambia bastante. Con `min_cluster_size=15` y `min_samples=3` el ruido baja de 894 a 424, los grupos pasan de 10 a 14 y la silueta sube de 0.0294 a 0.1160, el mejor valor que obtuve para este algoritmo.

El renglón de `min_cluster_size=5` con `min_samples` por omisión me dio 0.8202, que es exactamente el score de la partición `y_2` de la práctica 1B.

7. Resultados

Los parámetros con los que corro cada algoritmo son los que salieron de los barridos: `KMeans(n_clusters=9)`, `DBSCAN(eps=1.5, min_samples=20)`, `HDBSCAN(min_cluster_size=15, min_samples=3)` y `AgglomerativeClustering(n_clusters=9)` con enlace ward.

```
import pandas as pd

y_km = KMeans(n_clusters=9, n_init=10, random_state=0).fit_predict(X)
y_db = DBSCAN(eps=1.5, min_samples=20).fit_predict(X)
y_hd = HDBSCAN(min_cluster_size=15, min_samples=3,
               copy=True).fit_predict(X)
y_ag = AgglomerativeClustering(n_clusters=9).fit_predict(X)
etiquetas = [y_km, y_db, y_hd, y_ag]

res = {"algoritmos": ["k-means", "DBSCAN", "HDBSCAN",
                    "AgglomerativeClustering"],
      "intra/inter": [intra_inter(X, y, 500) for y in etiquetas],
      "silueta":     [silhouette_score(X, y) for y in etiquetas]
      }
output = pd.DataFrame(res)
output
```

	algoritmos	intra/inter	silueta
0	k-means	0.718899	0.189253
1	DBSCAN	0.842054	0.144041
2	HDBSCAN	0.818718	0.115991
3	AgglomerativeClustering	0.732730	0.180620

Las distancias entre renglones en la columna `intra/inter` son chicas: 0.0138 entre k-means y AgglomerativeClustering, 0.0233 entre HDBSCAN y DBSCAN. Los 500 pares salen de una sola semilla, así que antes de leer esa columna como un orden repito el muestreo con 200 semillas distintas y miro cuántas veces se sostiene cada comparación. Reproduzco el muestreo aparte para poder cambiarle la semilla.

```
def ii_por_fuera(datos, etiq, semilla):
    np.random.seed(semilla)
    p = np.random.randint(len(datos), size=(500, 2))
    d = np.sqrt(((datos[p[:, 0]] - datos[p[:, 1]]) ** 2).sum(axis=1))
    m = etiq[p[:, 0]] == etiq[p[:, 1]]
    return d[m].mean() / d[~m].mean()

tabla = np.array([[ii_por_fuera(X, y, s) for y in etiquetas]
                  for s in range(200)])
dif_hd = tabla[:, 1] - tabla[:, 2]
dif_km = tabla[:, 3] - tabla[:, 0]

print("200 semillas de muestreo")
print(" " * 21 + "veces menor    dif media    sd")
for nombre, dif in [("HDBSCAN contra DBSCAN", dif_hd),
                    ("k-means contra Agglom", dif_km)]:
    print(f" {nombre:22s}{100 * (dif > 0).mean():6.0f}%"
          f"{dif.mean():12.4f}{dif.std(ddof=1):8.4f}")
```

200 semillas de muestreo

	veces menor	dif media	sd
HDBSCAN contra DBSCAN	67 %	0.0118	0.0249
k-means contra Agglom	72 %	0.0076	0.0133

8. ¿Las dos métricas coinciden en el mejor algoritmo?

La columna intra/inter no ordena la tabla. Cambiando la semilla del muestreo, k-means queda por debajo de AgglomerativeClustering el 72 % de las veces y HDBSCAN por debajo de DBSCAN el 67 %, y las dos diferencias medias —0.0076 y 0.0118— son menores que su propia desviación. Con 500 pares esta métrica no separa ninguno de los dos pares, y el que me pareció menos firme al ver la tabla resultó ser el más estable de los dos.

Así que sí coinciden en el mejor: k-means tiene la intra/inter más baja (0.7189) y la silueta más alta (0.1893), y AgglomerativeClustering queda segundo en las dos. Pero esa coincidencia la sostiene la silueta, que es la que no muestrea nada y devuelve el mismo número en cada corrida. En el tercer y cuarto lugar las dos se contradicen de frente: la silueta pone a DBSCAN (0.1440) sobre HDBSCAN (0.1160) y la intra/inter los invierte, esto último sin firmeza.

Queda la salvedad de que fue la silueta la que eligió los parámetros de los cuatro. Si hubiera elegido con la intra/inter, de mis mismas tablas saldría k-means con $k = 50$ en 0.5736, Agglomerative con $k = 20$ en 0.6534, y HDBSCAN y DBSCAN empatados en 0.7851 y 0.7863: casi el mismo orden, pero con particiones de cincuenta y de veinte grupos que ya no describen un conjunto de diez dígitos.

9. Comprobaciones

Lo que sigue lo agregué por mi cuenta, con los mismos parámetros de la tabla.

Ver 0.8202 en el barrido de HDBSCAN me hizo sospechar que las cuatro particiones que me dieron en 1B habían salido de estos algoritmos. Probé combinaciones cercanas a las de mis barridos hasta que algunas coincidieron dato por dato.

```
candidatos = []
for eps in [1.4, 1.5]:
    for ms in [2, 3, 5]:
        candidatos.append((f"DBSCAN({eps}, {ms})",
                           DBSCAN(eps=eps, min_samples=ms).fit_predict(X)))
for mcs in [3, 5, 8]:
    candidatos.append((f"HDBSCAN({mcs})",
                       HDBSCAN(min_cluster_size=mcs,
                                copy=True).fit_predict(X)))
for k in [5, 9, 10]:
    candidatos.append((f"Agglomerative({k})",
                       AgglomerativeClustering(n_clusters=k
                                                ).fit_predict(X)))
for k in [5, 10]:
    candidatos.append((f"KMeans({k})",
                       KMeans(n_clusters=k, n_init=10,
                               random_state=0).fit_predict(X)))

guardadas = [np.load(f"y_{i}.npy") for i in range(4)]
for nombre, cand in candidatos:
    coincide = [f"y_{i}" for i, g in enumerate(guardadas)
                if np.array_equal(cand, g)]
    print(f" {nombre:20s} {coincide[0] if coincide else '-'}")

print()
for i, g in enumerate(guardadas):
    print(f" y_{i}: intra/inter {intra_inter(X, g, 500):.4f}")

print()
print("etiquetas barajadas, 3 repeticiones por algoritmo")
rng = np.random.default_rng(0)
for nombre, y in zip(res["algoritmos"], etiquetas):
    ctl = [intra_inter(X, rng.permutation(y), 500) for _ in range(3)]
    print(f" {nombre:24s}" + " ".join(f"{v:.4f}" for v in ctl))
```

DBSCAN(1.4, 2)	-
DBSCAN(1.4, 3)	-
DBSCAN(1.4, 5)	-
DBSCAN(1.5, 2)	y_1
DBSCAN(1.5, 3)	-
DBSCAN(1.5, 5)	-
HDBSCAN(3)	-
HDBSCAN(5)	y_2

```

HDBSCAN(8)          -
Agglomerative(5)    y_3
Agglomerative(9)    -
Agglomerative(10)   -
KMeans(5)           -
KMeans(10)          -

```

```

y_0: intra/inter 0.7405
y_1: intra/inter 0.9082
y_2: intra/inter 0.8202
y_3: intra/inter 0.8414

```

etiquetas barajadas, 3 repeticiones por algoritmo

```

k-means          1.0258  0.9977  1.0312
DBSCAN           1.0226  0.9992  0.9813
HDBSCAN          0.9854  0.9697  1.0512
AgglomerativeClustering 1.0361  0.9849  0.9861

```

De los catorce candidatos, tres dieron en el blanco: y_1 es DBSCAN(`eps=1.5`, `min_samples=2`), y_2 es HDBSCAN(`min_cluster_size=5`) y y_3 es AgglomerativeClustering(`n_clusters=5`). y_0 no salió de ninguno de los que probé. Los scores que imprimo debajo son los mismos que reporté en la práctica anterior, que es lo que quería confirmar.

Barajar las etiquetas conservando los tamaños de grupo deja los doce valores entre 0.9697 y 1.0512, es decir en 1. Cada score queda entonces medido contra su propio reparto barajado, y ahí la separación de DBSCAN es la más pobre de las cuatro: 0.8421 contra un control de 1, con el reparto al azar del principio en 0.9822.

```

print("descartando los datos marcados con -1")
for nombre, y in zip(res["algoritmos"], etiquetas):
    if not (y == -1).any():
        continue
    m = y != -1
    print(f" {nombre:8s} quedan {m.sum():4d} datos      "
          f"intra/inter {ii_por_fuera(X[m], y[m], 0):.4f}      "
          f"silueta {silhouette_score(X[m], y[m]):.4f}")

```

descartando los datos marcados con -1

```

DBSCAN   quedan 1330 datos   intra/inter 0.6282   silueta 0.2509
HDBSCAN  quedan 1373 datos   intra/inter 0.5949   silueta 0.2210

```

Sobre los 1330 datos que DBSCAN sí agrupó su intra/inter es 0.6282 y su silueta 0.2509; HDBSCAN sobre sus 1373 datos da 0.5949 y 0.2210. Los dos superan a k-means medido en el conjunto entero. Las cifras no son comparables con las de la tabla, porque salen de menos datos y de los más fáciles de cada método, pero fijan de qué tamaño es el efecto de la decisión sobre el -1.

10. Conclusión

k-means en nueve grupos es el mejor agrupamiento de los cuatro en las dos métricas a la vez, y ahí termina la coincidencia. La intra/inter, con los 500 pares que pide el enunciado, no alcanza a ordenar ni a k-means contra AgglomerativeClustering ni a HDBSCAN contra DBSCAN: en 200 remuestreos cada comparación se sostiene alrededor de dos tercios de las veces y su diferencia media no llega a una desviación. Lo que sí queda es la contradicción entre las dos métricas en el tercer y cuarto lugar, y se sostiene por el lado de la silueta, que no muestrea.

Los parámetros por omisión no habrían servido para nada de esto. DBSCAN con `eps=0.5` marca los 1797 datos como ruido, y HDBSCAN con todo por omisión da 0.0719 de silueta contra el 0.1160 que saqué al separar `min_samples` de `min_cluster_size`.

Y ese tercer y cuarto lugar miden en buena parte otra cosa que la calidad de los grupos. Sobre los datos que sí agruparon, DBSCAN y HDBSCAN superan a k-means en las dos métricas. Lo que los hunde en la tabla no es que agrupen mal lo que agrupan, sino que dejan 467 y 424 datos fuera y esos datos entran a las métricas como un grupo más.